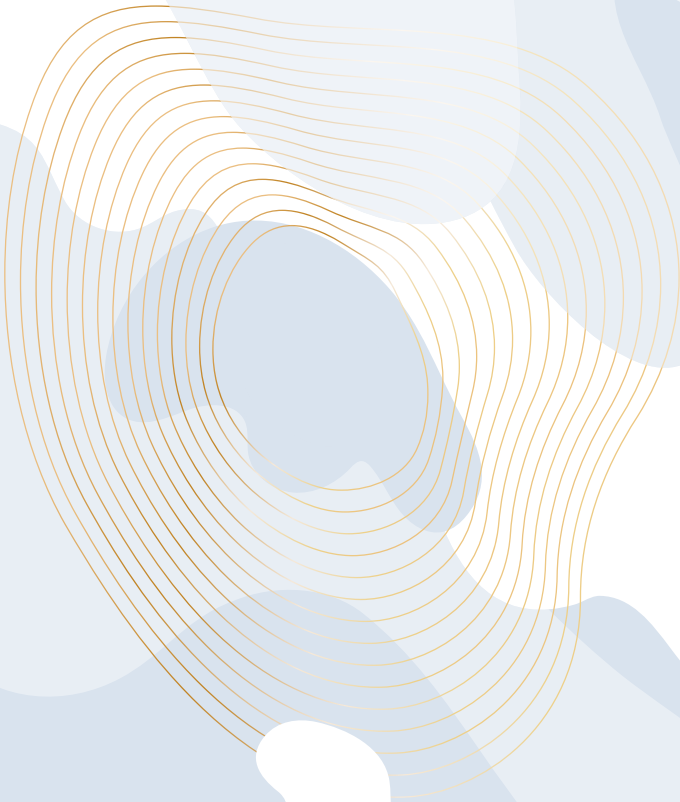


JAVA NOTES



Created by: Adipta Basu

Core Java Fundamentals

What is Java? JVM, JDK, JRE

JDK is a software development kit used to build Java applications. it contains the JRE and a set of development tools.

- Includes compiler (javac), debugger, and utilities like jar and javadoc.
- Provides the JRE, so it also allows running Java programs.
- Required by developers to write, compile, and debug code.

JRE provides an environment to run Java programs but does not include development tools. It is intended for end-users who only need to execute applications.

- Contains the JVM and standard class libraries.
- Provides all runtime requirements for Java applications.
- Does not support compilation or debugging.

Working of JRE:

1. **Class Loading**: Loads compiled .class files into memory.
2. **Bytecode Verification**: Ensures security and validity of bytecode.
3. **Execution**: Uses the JVM (interpreter + JIT compiler) to execute the instructions and make system calls.

JVM is the core execution engine of Java. It is responsible for converting bytecode into machine-specific instructions.

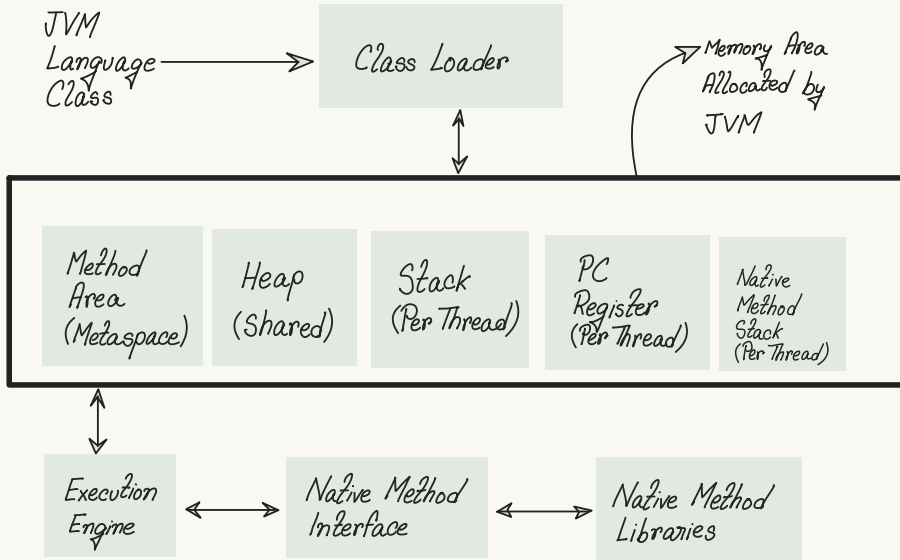
- ↳ Part of both JDK and JRE.
- Performs memory management and garbage collection.
- Provides portability by executing the same bytecode on different platforms.

Note:

- JVM implementations are platform-dependent.
- Bytecode is platform-independent and can run on any JVM.
- Modern JVMs rely heavily on Just-In-Time (JIT) compilation for performance.

Working of JVM:

1. **Loading**: Class loader loads bytecode into memory.
2. **Linking**: Performs verification, preparation, and resolution.
3. **Initialization**: Executes class constructors and static initializers.
4. **Execution**: Interprets or compiles bytecode into native code.



JDK vs JRE vs JVM

Aspect	JDK	JRE	JVM
Purpose	Used to develop Java applications	Used to run Java applications.	Executes Java bytecode
Platform Dependency	Platform-dependent ² (OS specific)	Platform-dependent ² (OS specific)	JVM is OS-specific but bytecode is platform-independent
Includes	JRE + Development tools (javac, debugger, etc.)	JVM + Libraries	ClassLoader, JIT Compiler, Garbage Collector

How Java Code Runs

Java follows the model:

Write Code → Compile → Generate Bytecode → Execute on JVM

This is what gives Java its platform independence.

- 1. Writing Source Code:** Create a file such as 'MyClass.java', this contains human-readable Java code.
- 2. Compilation:** The java compiler compiles the '.java' file.
`javac MyClass.java` It doesn't generate machine code. Instead, it produces a '.class' file. `MyClass.class`
This .class file contains bytecode, which is:
 - Platform-independent
 - Verified and safe
 - Executed by the JVM
- 3. Class Loading (by ClassLoaders):** Java uses a hierarchical class loading:
 - Bootstrap ClassLoader
 - Loads core Java classes (`java.lang.*`, `java.util.*`).
 - Part of the JVM, written in native code.
 - Platform/Extension ClassLoader
 - Loads extension or platform modules.
 - Acts as a bridge between bootstrap and application loaders.
 - Application/System ClassLoader
 - Loads user classes from the classpath.
 - Responsible for loading the project's .class files.

Each loader delegates to its parent first, ensuring security and consistency.
- 4. Bytecode Verification:** Before execution, the JVM verifies bytecode to ensure:
 - No illegal bytecode instructions
 - No stack corruption
 - No invalid type casts
 - No harmful memory access

5. **Execution by JVM**: The JVM uses two execution strategies:

— **Interpreter**

- Executes bytecode instruction by instruction.
- Provides fast startup but slower repeated execution.

— **JIT Compiler (Just-In-Time)**

- Converts frequently used bytecode ('hotspots') into native machine code.
- Stores compiled code in memory for reuse.
- Greatly boosts performance during runtime.

The JVM dynamically chooses between interpreter and JIT.

6. **Runtime Execution (JVM Internals)**: The JVM uses several internal components:

— **ClassLoader Subsystem**

- Load classes when required.

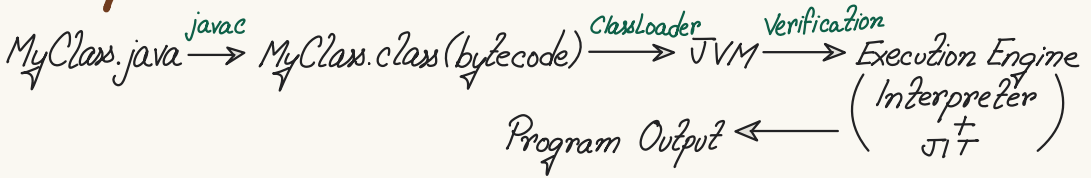
— **Execution Engine**

- Interpreter
- JIT Compiler
- Garbage Collector

— **Runtime Data Areas**

- Heap (Objects)
- Stack (method calls, local variables)
- Method Area (class metadata)
- PC Register (next instruction)
- Native Method Stack

Compilation & Execution Flow



Important Key Points

- Java is platform-independent because of bytecode + JVM, not the language.
- JIT compiler provides near-native performance after warm-up.
- ClassLoader hierarchy ensures security and prevents malicious class replacement.
- Bytecode verification unsafe or corrupted code from running.

Java Architecture Overview

Java architecture explains how different components work together to run a Java program efficiently and securely.

At a high level, Java architecture consists of:

- Java Source Code
- Runtime Environment
- Compiler
- Native System
- JVM

Main Components of Java

1. Java Compiler (javac)

- Converts java source code into .class bytecode.
- Performs syntax checking and basic validations.
- Does not generate machine code.

2. Java Virtual Machine (JVM)

- Core of Java architecture.
- Executes bytecode generated by the compiler.
- Platform-dependent (Different JVM for Windows, Linux, macOS).
- Provides memory management, security, and performance optimizations.

3. Java Runtime Environment (JRE)

- Provides libraries and runtime support.
- Required to run Java applications.
- Includes JVM, Core class libraries, supporting files

4. Java Development Kit (JDK)

- Used to develop Java applications.
- Includes JRE, Compiler (javac), Debugger, Tools (javadoc, jar, etc.)

Internal Architecture of JVM

The JVM itself is divided into multiple subsystems:

1. ClassLoader Subsystem

- Loads .class files into memory.
- Follows parent delegation model.
- Prevents loading of malicious classes.

2. Runtime Data Areas

Memory areas used during program execution:

- Heap
 - Stores objects and class instances.
 - Shared across threads.
 - Managed by Garbage Collector.
- Stack
 - Stores method calls and local variables.
 - Each thread has its own stack.
- Method Area
 - Stores class-level data:
 - Class metadata
 - Method bytecode
 - Static variables
- PC Register
 - Stores address of the current instruction.
- Native Method Stack
 - Use for native (non-Java) methods.

3. Execution Engine

Responsible for executing bytecode. Includes:

- Interpreter — executes line by line
- JIT Compiler — converts bytecode to native code
- Garbage Collector — frees unused memory.

Security in Java Architecture

Java provides built-in security using:

- Bytecode verification
- ClassLoader restrictions
- Security Manager (legacy/optional in modern Java)

This prevents:

- Memory corruption
- Unauthorized access
- Execution of unsafe code

Platform Independence Explained

- Java source code → bytecode (platform-independent)
- JVM converts bytecode → machine code (platform-dependent)
- Same bytecode runs on any OS with a compatible JVM.

Points to Remember

- JVM is platform-dependent; Java bytecode is not.
- JRE is required to run Java programs.
- JDK is required to develop Java programs.
- Memory management is automatic via Garbage Collection.
- JVM architecture ensures performance, security, and portability.

Java Memory Model

The Java Memory Model (JMM) defines how memory is allocated, used, and shared during program execution.

Each Java program runs inside a JVM, which manages memory automatically.

Main Memory Areas in JVM

Java memory is divided into the following runtime data areas:

1. **Heap Memory**: Used to store objects and class instances.

Key Characteristics:

- Shared across all threads
- Created when JVM starts
- Managed by Garbage Collector
- Can grow or shrink dynamically.

Stores:

- Objects (**new** keyword)
- Instance Variables

Example: `Student s = new Student();`

→ reference → stack
→ object → heap

2. **Stack Memory**: Stores method calls and local variables.

Key Characteristics:

- Each thread has its own stack.
- Memory is allocated in stack frames.
- Fast access (LIFO)
- Automatically freed when method ends.

Stack Frame Contains:

- Local variables
- Method parameters
- Return address
- Operand stack

Example: `void add() {
 int x = 10;
}`

→ stack

3. **Method Area**: Stores class-level information.

Stores:

- Class metadata
- Method bytecode
- Static variables
- Constant pools

Key Characteristics:

- Shared across threads.
- Created when class is loaded
- Part of heap in modern JVMs (Metaspace)

4. **PC (Program Counter) Register**: Keeps track of the current instruction executed.

Key Characteristics:

- One PC register per thread.
- Stores address of the next bytecode instruction
- Helps in thread switching

5. **Native Method Stack**: Used for native (non-Java) methods.

Key Characteristics:

- Handles calls to C/C++ code
- Used when Java interacts with OS or hardware

Memory Interaction Example:

```
class Test
{
    static int count=0;
    void run()
    {
        int x=5;
    }
}
```

Item	Memory Area
Test class metadata	Method Area
count	Method Area
run() call	Stack
x	Stack
Object of Test	Heap

Common Memory Errors

StackOverflowError

- Deep or infinite recursion
- Stack memory exhausted

OutOfMemoryError

- Heap or Metaspace exhausted
- Too many objects or large objects

Thread Safety & Memory

- Stack memory → thread-safe (each thread has its own)
- Heap memory → shared → requires synchronization

Points to Remember

- Objects live in Heap; references live in Stack
- Static variables live in Method Area
- Stack memory is faster but limited
- Heap memory is slower but larger
- Garbage Collection works on Heap memory

Java Class Structure

A class is the fundamental building block of Java. It represents a blueprint defining:

- State → variables
- Behavior → methods

Objects are created from classes.

Basic Class Structure

```
class ClassName
{
    // 1. Fields (variables)
    // 2. Static Block
    // 3. Instance Block
    // 4. Constructors
    // 5. Methods
}
```

Order is not mandatory (except blocks), but this is the recommended structure.

1. **Fields (Variables)**: Fields represent the state of an object or class.

Types of Fields:

- Instance variables → per object
- Static variables → shared across class

```
class Student
{
    int id; // instance variable
    static String school; // static variable
}
```

Memory:

- Instance variables → Heap
- Static variables → Method Area

2. **Methods**: Methods define the behavior of a class.

```
void study()  
{  
    System.out.println("Studying");  
}
```

Method Components:

- Access modifier
- Return type
- Method name
- Parameters
- Method body

Types of Methods:

- Instance methods → operate on object data
- Static methods → operate on class-level data
- Abstract methods → declared, not implemented
- Final methods → cannot be overridden

3. **Constructors**: Constructors are used to initialize objects.

Key Characteristics:

- Same name as class
- No return type (not even void)
- Automatically invoked during object creation

```
class Student
```

```
{  
    int id;  
    Student(int id)  
    {  
        this.id = id;  
    }  
}
```

Types of Constructors

- Default constructor (no arguments)
- Parameterized constructor

Constructor Chaining:

- `this()` → calls another constructor in same class
- `super()` → call parent class constructor

4. **Static Initialization Block**: Executed once, when the class is loaded into memory.

```
static  
{  
    System.out.println("Class Loaded");  
}
```

Used for:

- Initializing static variables
- One-time setup logic

5. *Instance Initialization Block*: Executed everytime an object is created, before the constructor.

```
{  
    System.out.println("Object created");  
}
```

Used for:

- Common initialization code shared by all constructors

Execution Order in a Java Class:

1. Static variables
2. Static blocks
3. Instance variables
4. Instance initialization blocks
5. Constructor
6. Methods

Access Modifiers in Class Members

Modifier	Within Class	Same Package	Subclass	Everywhere
private	✓	✗	✗	✗
default	✓	✓	✗	✗
protected	✓	✓	✓	✗
public	✓	✓	✓	✓

Important Rules & Notes

- Only one public class per java file.
- File name must match the public class name
- Static members belong to the class, not objects
- Constructors are not inherited
- 'this' refers to current object
- 'super' refers to parent class

Data Types

Categories of Data Types in Java

A data type defines:

- What kind of data a variable can hold.
- How much memory it uses.
- What operations can be performed on it.

Java is a strongly typed language, meaning every variable must have a declared type.

Main Categories of Data Types

Java data types are divided into two broad categories:

1. **Primitive Data Types**: Primitive data types store actual values, not references.

Key Characteristics:

- Fixed Memory size
- Fast access
- Stored directly in memory
- Cannot be null
- Not objects

Primitive Types:

- byte
- short
- int
- long
- float
- double
- char
- boolean

Primitive types are the building blocks of Java

2. **Non-Primitive (Reference) Data Types**: Reference data types store memory addresses (references), not actual data.

Key Characteristics:

- Can store complex data
- Stored in Heap memory
- Can be null
- Have method and properties
- Size is not fixed

Examples:

- Classes
- Objects
- Arrays
- Interfaces
- Strings
- Enums

`String name = "Java";`

`name` → reference stored in Stack.

`"Java"` → object stored in Heap.

Primitive vs Reference – Difference

Feature	Primitive	Reference
Stores	Actual Value	Memory address
Null Allowed	✗ No	✓ Yes
Size	Fixed	Variable
Methods	✗ No	✓ Yes
Stored in Heap	✗ No	✓ Yes

Memory Perspective (Important)

```
int x=10;  
String s="Java";
```

Variables `x`, `s` (reference) are stored in Stack Memory.
"Java" (string object) is stored in Heap Memory.

Why Java Separates Data Types

- Improves performance (primitives are faster)
- Reduces memory overhead
- Allows object-oriented features via references
- Enables better garbage collection

Important Notes to Remember

- Collections work only with reference types.
- Primitives must be wrapped using wrapper class.
- Arrays are reference types even if they store primitives
- Java does not support pointers directly.

Primitive Data Types

Primitive data types are the basic building blocks of data in Java. They store actual values, not references.

Key Characteristics of Primitive Types:

- Fixed size
- Fast performance
- Stored directly in memory
- Cannot be null.
- Not objects (no methods)

Primitive Data Types - Overview

Type	Size	Default	Range	Notes
byte	1 byte	0	-128 to 127	Used in I/O, streams
short	2 bytes	0	-32,768 to 32,767	Rarely Used
int	4 bytes	0	-2^{31} to $2^{31}-1$	Most common integers
long	8 bytes	0L	-2^{63} to $2^{63}-1$	Use L suffix
float	4 bytes	0.0f	~6-7 decimal digits	Use f suffix
double	8 bytes	0.0	~15 decimal digits	Default decimal type
char	2 bytes	'\u0000'	0 to 65,535	Unicode
boolean	JVM-dependent	false	true/false	No numeric value

Integer Data Types

byte:

- 8-bit signed integer
- Memory efficient
- Common in file handling and streams

byte b = 100;

short:

- 16 bit signed integer
- Limited usage in modern Java.

short s = 1000;

int:

- 32-bit signed integer
- Default choice for whole numbers

int count = 10;

long:

- 64-bit signed integer
- Used for very large numbers

long population = 8_000_000_000L;

Floating-Point Data Types

float:

- Single-precision
- Less accurate
- Faster but rarely used.

float price = 10.5f;

double:

- Double-precision
- More accurate
- Default for decimal values.

double pi = 3.14159;

Character Data Type(char)

- Stores a single Unicode character
- 16-bit unsigned
- Supports international characters

char c = 'A';

char symbol = '\u20B9'; // ₹

Boolean Data Type

- Represents logical values
- Only `true` or `false`
- Not convertible to/from numbers

`boolean isValid = true;`

Memory Behavior of Primitives

`int x = 10;`

- `x` is stored directly in stack memory
- No object creation
- Very fast access

Important Rules & Notes

- Size of primitives is fixed across platforms
- Arithmetic on smaller types (byte, short) promotes to int.
- Floating-point numbers may have precision issues
- Use `BigDecimal` for financial calculations

Interview Key Points

- `int` is preferred over `byte`/`short`
- `double` is preferred over `float`
- `char` uses Unicode, not ASCII
- `boolean` size is JVM-dependent

Type Conversion in Java

Type conversion is the process of converting one data type into another. Java supports two kinds of type conversion:

1. Widening (Implicit)
2. Narrowing (Explicit/Casting)

Widening Conversion (Implicit)

Widening conversion happens automatically.

Characteristics:

- No data loss
- Safe conversion
- Performed by compiler
- Smaller $\rightarrow$ large data type

Widening Order:

byte $\rightarrow$ short $\rightarrow$ int $\rightarrow$ long $\rightarrow$ float $\rightarrow$ double

Example:

```
int x = 10;  
double d = x;
```

Narrowing Conversion (Explicit)

Narrowing conversion must be done manually.

Characteristics:

- Possible data loss
- Explicit cast required
- Larger $\rightarrow$ smaller data type

Example:

```
double d = 10.75;  
int x = (int) d; // x = 10
```

Decimal part is truncated, not rounded.

Primitive Casting Examples

```
int a = 130;  
byte b = (byte) a;
```

Result $\rightarrow$ $b = -126$
 $\rightarrow$ This happens due to overflow

Why Overflow Happens

byte range: -128 to 127

It will just keep the 1st byte, or 8 bits

Java doesn't throw an error—it wraps the value

char and numeric conversion

```
char c = 'A';  
int x = c; // x = 65  
int y = 66;  
char d = (char) y; // d = 'B';
```

char is internally treated as an unsigned integer.

Type Promotion in Expressions

Java promotes similar types during arithmetic.

Example:

```
byte a = 10;  
byte b = 20;  
byte c = a + b; // ❌ compile error
```

Correct way:

```
byte c = (byte) (a + b);
```

Reason: $a + b$ is promoted to int

Integer vs Floating Division

```
int a = 5 / 2; // 2  
double b = 5 / 2; // 2.0  
double c = 5 / 2.0; // 2.5
```

At least one operand must be floating point

Important Casting Rules

- boolean cannot be cast to/from any other type.
- Object casting works only with inheritance
- Widening is always safe; narrowing is risky
- Casting doesn't change the underlying value, only representation

Common Pitfalls

- Data loss during narrowing
- Overflow and underflow
- Unexpected integer division
- Forgetting explicit cast

Interview Key Points

- Widening is implicit, narrowing is explicit
- Smaller integer types are promoted to int.
- Casting floating integers truncates value
- boolean has no numeric casting

Wrapper Class

Wrapper classes are object representation of primitive data types. They allow primitives to be:

- Used in Collections
- Used with Generics
- Treated as objects

Why Wrapper Classes Exist

Java is object-oriented, but primitives are not objects. Wrapper classes solve this by:

- Wrapping primitive values inside objects
- Providing utility methods.
- Allowing null values

Primitive ↔ Wrapper Mapping

Primitive	Wrapper Class
byte	Byte
short	Short
int	Integer
long	Long
float	Float
double	Double
char	Character
boolean	Boolean

Key Characteristics:

- Stored as objects in Heap memory
- Can be null
- Immutable (value cannot change)
- Belong to java.lang package

Creating Wrapper Objects

1. Using Constructor (Deprecated)

```
Integer a = new Integer(10);
```

Deprecated → avoid using

2. Using valueOf() (Recommended)

```
Integer b = Integer.valueOf(10);
```

• Uses caching • Better performance

3. From String

```
Integer c = Integer.valueOf("123");
```

• Throws `NumberFormatException` if invalid.

4. Extracting Primitive Value

```
Integer x = 10;
```

```
int y = x.intValue();
```

5. Utility Methods (Very Important)

```
Integer.parseInt("100"); // int
```

→ returns primitive

```
Integer.valueOf("100"); // Integer
```

→ wrapper

Why Collections Require Wrappers

```
List<int> list; // X Not allowed
```

```
List<Integer> list; // ✓ Allowed
```

Generics work only with reference types, not primitives.

Immutability of Wrapper Classes

```
Integer a = 10;  
a = a + 5;
```

A new object is created; original value is unchanged.

Comparison Pitfall

```
Integer a=100;  
Integer b=100;  
System.out.println(a==b); // true  
Integer x=200;  
Integer y=200;  
System.out.println(x==y) // false
```

Reason:

- Integer cache range: -128 to 127
- Use `.equals()` for value comparison.

Primitive variable / Wrapper references are stored in Stack. Wrapper object in Heap.

Important Rules to Remember

- Wrapper classes are immutable
- Use `equals()`, not `==`
- Avoid constructors
- Required for Collection and Streams
- Caching improves performance

Interview One-Liners

- Wrapper classes convert primitives into objects
- Need for generics and collections
- `parseXXX()` returns primitive, `valueOf()` returns wrapper
- Wrapper classes are immutable.

Autoboxing & Unboxing

Autoboxing and unboxing are automatic conversions between:

- Primitive types
- Their corresponding wrapper classes

Introduced in Java 5.

Autoboxing Primitive → Wrapper Obj

```
int a = 10;
```

```
Integer x = a; // autoboxing
```

→ What the compiler actually does:

```
Integer x = Integer.valueOf(a);
```

Unboxing Wrapper Obj → Primitive

```
Integer x = 20;
```

```
int b = x;
```

→ What the compiler actually does:

```
int b = x.intValue();
```

Why Autoboxing Exists

- To simplify code
- To allow primitives to work with:
 - Collections
 - Generics
 - Streams
- To remove boilerplate conversions

Autoboxing in Collections

```
List<Integer> list = new ArrayList<>();  
list.add(10); //autoboxing  
int x = list.get(0); //unboxing
```

Behind the scenes:

- $10 \rightarrow \text{Integer.valueOf}(10)$
- $\text{Integer} \rightarrow \text{intValue}()$

Performance Consideration

```
for (int i=0; i<1000; i++) {  
    Integer x=i; //repeated boxing  
}
```

- Creates many objects
- Slower than primitives
- Avoid in tight loops

Autoboxing with Operators

```
Integer a=10;  
Integer b=20;  
Integer c=a+b;
```

a, b are unboxed to `int`, then the addition happens. After which the results are boxed back into `Integer`.

Best Practices

- Prefer primitives for calculations.
- Avoid unnecessary boxing/unboxing
- Always check for null before unboxing
- Use `equals()` for wrapper comparison

Integer Cache & Common Pitfalls

Java caches certain wrapper objects to improve performance. This mainly affects Integer (and some other wrappers).

What is Integer Cache?

Java maintains a cache of Integer object for values: -128 to 127 (inclusive)
These cached objects are reused, not recreated.

Why Integer Cache Exists

- These values are commonly used
- Reduces memory usage
- Improves performance
- Implemented inside Integer.valueOf()

How Caching Works

```
Integer a = 100;  
Integer b = 100;  
System.out.println(a==b); // true
```

Why?

- Both a and b point to the same cached object.

```
Integer a = 200;  
Integer b = 200;  
System.out.println(a==b); // false
```

Why?

- 200 is outside cached range
- Two separate objects created.

Other Wrapper Caches

Wrapper	Cached Range
Byte	Entire Range
Short	-128 to 127
Integer	-128 to 127
Long	-128 to 127
Character	0 to 127
Boolean	true/false

Memory Perspective

Scenario	Result
Cached Integer	Same object reused
Non-cached Integer	New object created
== on wrappers	Reference comparison
.equals()	Value comparison

Best Practices

- Never use == for wrapper comparison.
- Use primitives when possible
- Be cautious with autoboxing in loops
- Check for null before unboxing.

Literals in Java

A *literal* is a fixed value written directly in the source code.

`int x=10;` // 10 is a literal

Why Literals Matter

- Define the type and value at compile time
- Affect memory usage and type inference
- Incorrect literals cause compile-time errors

1. Integer Literals

Int Literals

By default, integer literals are of type `int`.

`int a=100;`

Long Literals

- Must end with 'L' or 'l' (prefer 'L')

`long population=8_000_000_000L;`

Without L → compile-time error (out of `int` range).

Numeric Separators (_)

Improves readability.

Rules:

- Cannot be at start or end
- Cannot be next to decimal point.

`int salary=1_00_000;`

`long distance=9_223_372_036_854_775_807L;`

2. Floating-Point Literals

Double Literals

- Default type for decimals

```
double pi = 3.14;
```

Float Literals

- Must end with 'f' or 'F'

```
float price = 10.5f;
```

Without f → compile-time error.

Scientific Notation

```
double d = 1.2e3; // 1200.0
```

3. Character Literals (char)

Can be represented in multiple ways:

All represent 'A'.

```
char a = 'A';  
char b = 65; // ASCII value  
char c = '\u0041'; // Unicode
```

4. Boolean Literals

Only two values:

```
boolean isValid = true;
```

```
boolean isDone = false;
```

- No numeric equivalents
- Cannot cast from/to numbers

5. Binary Literals (Java 7+)

Prefix: 0b or 0B

```
int x = 0b1010; // 10
```

6. Octal Literals

Prefix: 0

```
int x=010; //8
```

Rarely used, but important to know.

7. Hexadecimal Literals

Prefix: '0x' or '0X'

```
int x=0x1A; //26
```

Common use:

- Bit masks
- Colors
- Low-level programming

Default Types of Literals

Literal	Default Type
10	int
10L	long
10.5	double
10.5f	float
'A'	char
true	boolean

Important Notes

- Integer literals default to int
- Decimal literals default to double
- Use suffixes to change type
- Literals are resolved at compile time.

String & Text Processing Basics

In Java, text is represented using String-related classes from java.lang.

What is a String?

A String is a sequence of characters.

```
String s = "Java";
```

Key Properties:

- String is a class
- Strings are immutable
- Stored in String Pool (special heap area)

String Immutability

Once a String object is created, it cannot be changed.

```
String s = "Java";  
s.concat(" World");
```

Result:

- "Java" remains unchanged
- "Java World" is created as a new object.

Why Strings are Immutable

- Security (class loaders, file paths, URLs)
- Thread safety
- Caching in String Pool
- Better memory management

String Pool

```
String a = "Java";  
String b = "Java";
```

- Both a and b point to the same object
- Improves memory efficiency

```
String c = new String("Java");
```

- Forces creation of a new object outside pool.

String Creation Summary

Code	Objects Created
"Java"	1(pool)
new String("Java")	2(pool + heap)

StringBuilder

Mutable version of String (non-thread-safe).

```
StringBuilder sb = new StringBuilder("Java");  
sb.append(" World");
```

Use when:

- Single-threaded environment
- Frequent string modifications

✓ Same object modified

✓ Fast

✗ Not thread-safe

StringBuffer

Mutable and thread-safe version of String.

```
StringBuffer sb = new StringBuffer("Java");  
sb.append(" World");
```

Use when:

- Multi-threaded environment

✓ Thread-safe

✗ Slower than StringBuffer

String vs StringBuilder vs StringBuffer

Feature	String	StringBuilder	StringBuffer
Mutability	Immutable	Mutable	Mutable
Thread-safe	✓	✗	✓
Performance	Slow(modification)	Fast	Slower
Memory	Efficient(pool)	Heap	Heap

Performance Pitfall

```
String s = "";  
for(int i=0; i<1000; i++)  
{  
    s = s+i;  
}
```

✗ Creates many objects → slow

```
StringBuilder sb = new StringBuilder();  
for(int i=0; i<1000; i++)  
{  
    sb.append(i);  
}
```

✓ Only one object created

Important Notes

- Always use `.equals()` for String comparison
- Prefer `StringBuilder` for concatenation in loops
- Avoid `new String()` unless necessary
- Strings are immutable but references are not

Variables & Operators

Variable Types in Java

A variable is a named memory location used to store data. In Java, variables are classified based on where they are declared and their lifetime

Main Types of Variables

1. **Local Variables**: Declared inside a method, constructor, or block.

Characteristics:

- Scope limited to the block where declared
- Stored in stack memory
- No default value (must be initialized)
- Destroyed when method execution ends.

Example:

```
void add()  
{  
    int a = 5;  
    int b = 10;  
    System.out.println(a+b);  
}
```

2. **Instance Variables:** Declared inside a class but outside methods.

Example:

```
class Student
{
    int id;
    String name;
}
```

Characteristics:

- Each object has its own copy
- Stored in heap memory
- Default values are assigned automatically
- Exist as long as the object exists

Default Values:

Type	Default
int	0
double	0.0
boolean	false
object	null

3. **Static (Class) Variables:** Declared using the static keyword.

Example:

```
class Student
{
    static String schoolName;
}
```

Access:

```
Student.schoolName = "My School";
```

Characteristics:

- Belong to the class, not objects
- Single shared copy
- Stored in Method Area
- Created when class is loaded

Comparison of Variable Types

Feature	Local	Instance	Static
Declared in	Method/Block	Class	Class
Belongs to	Method	Object	Class
Memory	Stack	Heap	Method Area
Default Value	✗ No	✓ Yes	✓ Yes
Lifetime	Method execution	Object lifetime	Program lifetime

Common Pitfalls

- Using uninitialized local variables → compile-time error
- Accessing instance variables via class name → bad practice
- Overusing static variables → poor design

Best Practices

- Use local variables for temporary data
- Use instance variables for object state
- Use static variables for shared constants
- Prefer final static for constants

Scope & Lifetime of Variables

Scope defines where a variable can be accessed. Lifetime defines how long a variable exists in memory. Both depend on where and how the variable is declared.

Scope of Variables

When a local variable has the same name as an instance variable, it is called shadowing.

1. Local Variable Scope

- Accessible only inside the block where declared
- Includes:
 - Methods
 - Constructors
 - Loops
 - Conditional Blocks

2. Instance Variable Scope

- Accessible throughout the entire class
- Accessed via object reference

3. Static Variable Scope

- Accessible throughout the class
- Can be accessed using class name

Lifetime of Variables

1. Local Variable Lifetime

- Created when block execution starts
- Destroyed when block execution ends
- Stored in stack memory

2. Instance Variable Lifetime

- Created when object is created
- Destroyed when object becomes unreachable
- Memory managed by Garbage Collector

3. Static Variable Lifetime

- Created when class is loaded
- Exists until JVM shuts down

Scope and Lifetime Comparison

Variable Types	Scope	Lifetime
Local	Block	Method execution
Instance	Class (via object)	Object lifetime
Static	Class	Program lifetime

Best Practices

- Keep variable scope as small as possible
- Avoid shadowing unless necessary
- Use meaningful variable names
- Prefer local variables for temporary data

Operators in Java

An operator is a symbol that performs an operation on one or more operands.

Categories of Operators Covered

1. **Arithmetic Operators:** Used to perform mathematical operations.

Operator	Meaning	Example
+	Addition	$a + b$
-	Subtraction	$a - b$
*	Multiplication	$a * b$
/	Division	a / b
%	Modulus	$a \% b$

2. **Relational Operators:** Used to compare two values. Result is always boolean.

Operator	Meaning
>	Greater than
<	Less than
>=	Greater or equal
<=	Less or equal
==	Equal to
!=	Not equal to

3. **Logical Operators:** Used to combine boolean expressions.

Operator	Meaning
&&	Logical AND
	Logical OR
!	Logical NOT

Short-Circuit Behavior

false && expression // expression not evaluated
true && expression // expression evaluated

4. Bitwise Operators: Operate on binary representation of numbers.

Operator	Meaning
&	Bitwise AND
^	Bitwise XOR
~	Bitwise NOT
<<	Left shift
>>	Right shift
>>>	Unsigned right shift

```
int a=5; // 0101
int b=3; // 0011
System.out.println(a&b); // 1
System.out.println(a | b); // 7
```

Bit Shift Example:

```
int x=8; // 1000
System.out.println(x<<1); // 100002 = 1610
System.out.println(x>>1); // 1002 = 410
```

5. Ternary Operator: A shorthand for if-else.

Syntax: condition ? value-if-true : value-if-false;

Best Practices

- Use parenthesis for clarity
- Prefer && and || over & and | for conditions
- Use ternary only for simple conditions
- Avoid bitwise operators unless required.

Pre & Post Increment/Decrement

Increment (++) and decrement (--) operators increase or decrease a variable's value by 1.

Types of Increment/Decrement

1. Pre-Increment (++x): Value is incremented first, then used.

`int x = 5;`
`int y = ++x;`
Finally, it becomes $x = 6$
 $y = 6$

Step-by-step:

1. 'x' becomes 6
2. 'y' gets 6

2. Post-Increment (x++): Value is used first, then incremented.

`int x = 5;`
`int y = x++;`
Finally, it becomes $x = 6$
 $y = 5$

Step-by-step:

1. 'y' gets 5
2. 'x' becomes 6

3. Pre-Decrement (--x): Value is decremented first, then used.

`int x = 5;`
`int y = --x;`
Finally, it becomes $x = 4$
 $y = 4$

Step-by-step:

1. 'x' becomes 4
2. 'y' gets 4

2. Post-Decrement ($x--$): Value is used first, then decremented.

`int x = 5;`
`int y = x--;` $\xrightarrow[\text{becomes}]{\text{Finally, it}}$ $x = 4$
 $y = 5$

Step-by-step:

1. 'y' gets 5
2. 'x' becomes 4

Increment in Expressions (V.Imp)

`int a = 10;`
`int b = a++ + ++a;` $\xrightarrow[\text{becomes}]{\text{Finally, it}}$ $a = 12$
 $b = 22$

Step-by-step:

1. $a++ \rightarrow$ use 10, then $a = 11$
2. $++a \rightarrow$ increment $\rightarrow a = 12$, use 12
3. $b = 10 + 12 = 22$

Common Pitfalls

- Overusing $++$ in complex expressions
- Reducing readability
- Confusing evaluation order

Operator Precedence in Java

Operator precedence determines the order in which operators are evaluated in an expression. If multiple operators appear in an expression:

1. Higher precedence operators are evaluated first.
2. If precedence is same, associativity is used.

Why Operator Precedence Matters : $\text{int result} = 10 + 5 * 2;$
Result : $10 + (5 * 2) = 20$

Best Practices

- Use parenthesis to make intent clear
- Avoid complex expressions
- Never rely on memorizing full precedence
- Write readable code over clever code
- Parenthesis have highest precedence
- $*$ has higher precedence than $+$
- $\&\&$ evaluated before $\parallel$
- Assignment has lowest precedence

OOP in Java

Encapsulation in Java

Encapsulation is the process of bundling data (variables) and methods that operate on that data into a single unit (class), and restricting direct access to the data.
In simple terms: **Hide the data, expose controlled access.**

Why Encapsulation is Needed?

- Protects object state from invalid changes
- Improves code maintainability
- Enables controlled access
- Makes classes easier to use and modify

Benefits of Encapsulation

- Loose coupling
- Better control over data
- Easier debugging
- Flexibility to change implementation

How Encapsulation is Achieved

Encapsulation is implemented using:

1. Private data members
2. Public getter and setter method

```
class Student
{
    private int age;
    public int getAge()
    {
        return age;
    }
    public void setAge(int age)
    {
        if (age > 0)
            this.age = age;
    }
}
```

What is happening:

- age is hidden (private)
- Access is controlled via methods
- Validation logic is centralized

Data hiding is a part of encapsulation.

Inheritance in Java

Inheritance allows one class to acquire the properties and behavior of another class. In simple terms: *A child class reuses and extends a parent class.*

Key Terminology :

Term	Meaning
Parent Class	Superclass / Base class
Child Class	Subclass / Derived class
extends	Keyword used for inheritance

```
class Parent
{
    int x=10;
    void show()
    {
        System.out.println("Parent method");
    }
}
```

```
class Child extends Parent
{
    int y=20;
}
```

- Child inherits x and show() from Parent

Why Inheritance is Used

- Code reuse
- Extensibility
- Method overriding
- Supports polymorphism

Types of Inheritance in Java

✗ Supported:

1. Single Inheritance
2. Multilevel Inheritance
3. Hierarchical Inheritance

✗ Not Supported (with classes):

- Multiple Inheritance

1. Single Inheritance

```
class A { }  
class B extends A { }
```

2. Multilevel Inheritance

```
class A { }  
class B extends A { }  
class C extends B { }
```

3. Hierarchical Inheritance

```
class A { }  
class B extends A { }  
class C extends A { }
```

Inherited Members Rules:

Member	Inherited?
Public	✓
Protected	✓
Default	✓ Same package
Private	✗
Constructors	✗

✗ Why Java doesn't support multiple Inheritance (Classes)?

Reason:

- Diamond problem
- Ambiguity in method resolution

Java solves this using interfaces.

super Keyword

Used to:

- Access parent class members
- Call parent class constructor

```
class Child extends Parent  
{  
    Child()  
    {  
        super(); // calls Parent constructor  
    }  
}
```

Method Overriding

Child class provides its own implementation of a parent method.

```
class Parent
{
    void show()
    {
        System.out.println("Parent");
    }
}
```

```
class Child extends Parent
{
    @Override
    void show()
    {
        System.out.println("Child");
    }
}
```

Rules:

- Same method signature
- Access level cannot be reduced

Advantages

- Reduces code duplication
- Improves maintainability
- Enables runtime polymorphism

Important Rules

- Java supports only single inheritance for classes
- final methods cannot be overridden
- final classes cannot be inherited
- static methods are not overridden (they are hidden)

Polymorphism in Java

Polymorphism means "many forms". In Java, it allows the same method name to behave differently based on context. **One interface, many implementations.**

Types of Polymorphism in Java :

1. Compile-time Polymorphism

2. Runtime Polymorphism

Compile-Time Polymorphism

Method resolution happens at compile time.

Definition: Multiple methods with the same name, but different parameter list.

class Calculator

```
{
    int add(int a, int b)
    {
        return a+b;
    }
    double add(double a, double b)
    {
        return a+b;
    }
}
```

- Different parameter types
- Same method name

Also called method overloading

Overloading Rules:

- Method name must be same
- Parameter list must be different
- Return type alone cannot differ

Runtime Polymorphism

Method call is resolved at runtime, based on the object type.

```
class Parent
{
    void show()
    {
        System.out.println("Parent");
    }
}
class Child extends Parent
{
    @Override
    void show()
    {
        System.out.println("Child");
    }
}
```

`Parent p = new Child();`
`p.show(); // Child`

Dynamic Method Dispatch:
At runtime, JVM decides which overridden method to call, based on the actual object, not the reference type.

Important Overriding Rules

- Method signature must be same
- Access level cannot be reduced
- Final methods cannot be overridden
- Static methods are hidden, not overridden

How Runtime Polymorphism Works:

- Achieved using inheritance
- Uses method overriding
- Resolved using dynamic method dispatch

Polymorphism Example Explained: `Parent p = new Child();`

Methods → resolved using object type

Variable → accessed using reference type

Reference Type

Object Type

What is NOT Polymorphism?

- Variable overriding
- Static method overriding
- Constructor overriding

Polymorphism vs Overloading vs Overriding

Feature	Overloading	Overriding
Time	Compile-time	Runtime
Inheritance needed	✗	✓
Method signature	Different	Same
Binding	Static	Dynamic

Benefits of Polymorphism

- Loose coupling
- Code flexibility
- Easy extensibility
- Cleaner design

Abstraction in Java

Abstraction means hiding implementation details and showing only essential features to the user. Focus on what an object does, not how it does it.

Why Abstraction is Needed:

- Reduces complexity
- Improves maintainability
- Enables loose coupling
- Helps in large-scale design

Important Rules:

- Abstract classes can have constructors
- Interfaces cannot have constructors
- A class must implement all abstract methods
- Interfaces support multiple inheritance

How Abstraction is Achieved in Java:

Java provides two mechanisms:

1. Abstract Class
2. Interfaces

Benefits of Abstraction:

- Flexibility in implementation
- Loose coupling
- Easy code changes
- Better architecture

Abstract Class

An abstract class is a class that cannot be instantiated and may contain:

- Abstract methods (no body)
- Concrete methods (with body)
- Variables
- Constructors

Using Abstract Class

```
abstract class Vehicle
{
    abstract void move();
    void fuel()
    {
        System.out.println("Fueling");
    }
}
```

```
class Car extends Vehicle
{
    void move()
    {
        System.out.println("Car");
    }
}
```

Abstract Method:

- Declared using *abstract*
- Has no implementation
- Must be implemented by child class

Interface

An interface is a blueprint of a class that contains:

- Abstract methods (by default)
- default methods (Java 8+)
- static methods
- Constants (public static final)

```
interface Payment {  
    void pay();  
}
```

```
class CardPayment implements Payment {  
    {  
        public void pay()  
        {  
            System.out.println("Paying by Card");  
        }  
    }  
}
```

Abstract Class vs Interface

Feature	Abstract Class	Interface
Instantiation	✗	✗
Methods	Abstract + concrete	Abstract + default + static
Variables	instance variables	public static final only
Multiple inheritances	✗	✓
Constructors	✓	✗

Constructors & Chaining

A constructor is a special method used to initialize an object at the time of its creation. `ClassName obj = new ClassName();`

What is a Constructor?

- Name is same as the class
- No return type (not even void)
- Automatically invoked when an object is created
- Used to initialize variables

```
class Student2
{
    int id;
    Student2()
    {
        id = 1;
    }
}
```

Important Constructor Rules

- Only one of `this()` or `super()` allowed
- Must be the first line
- If neither is written, compiler inserts `super()`

Key Rules of Constructors:

- Constructor name = class name
- Can be overloaded
- Are not inherited
- Cannot be static, final, or abstract
- Called only once per object

Best Practices

- Use constructors for mandatory initialization
- Keep constructors lightweight
- Use constructor chaining to reduce duplication
- Prefer immutability where possible

Types of Constructors

1. Default Constructors

- Provided by compiler only if no constructor is defined
- Assigns default values

```
class Test
{
    int x;
}
```

compiler
adds

```
Test()
{
    x=0;
}
```

2. No-Argument Constructor

- Explicitly written by programmer
- Replaces default constructor

```
class Test
{
    Test()
    {
        System.out.println("No-arg constructor");
    }
}
```

3. Parameterized Constructor

```
class Student
{
    int id;
    Student(int id)
    {
        this.id = id;
    }
}
```

4. Constructor Overloading

Multiple constructors in the same class with different parameter lists.

```
class Box
{
    Box() {}
    Box(int x) {}
    Box(int x, int y) {}
}
```

- Same name
- Different parameters

'this' Keyword in Constructors

Used to:

- Refer to current object
- Resolve name conflicts
- Call another constructor in the same class

```
Student(int id)
{
    this.id = id;
}
```

Constructor Chaining

Calling one constructor from another constructor. Java supports chaining using:

- `this()` → same class
- `super()` → parent class

Constructor Execution Order

1. Parent Constructor
2. Child Constructor

Constructor Chaining Using this()

```
class Test
{
    Test()
    {
        this(10);
        System.out.println("No-arg");
    }
    Test(int x)
    {
        System.out.println("Parameterized");
    }
}
```

Rules:

- this() must be the first statement
- Avoid circular constructor calls.

Constructor Chaining Using super()

```
class Parent
{
    Parent()
    {
        System.out.println("Parent");
    }
}
class Child extends Parent
{
    Child()
    {
        super();
        System.out.println("Child");
    }
}
```

Notes:

- super() is added automatically if not written.
- Must be the first statement

Method Overloading & Overriding

Both are used to achieve polymorphism, but they work in very different ways.

High-Level Difference:

Concept	Overloading	Overriding
Type	Compile-time polymorphism	Runtime polymorphism
Inheritance required	✗ No	✓ Yes

Method Overloading

Definition: Method overloading means multiple methods with the same name in the same class, but with different parameter lists.

```
class Calculator
{
    int add(int a, int b)
    {
        return a+b;
    }
    int add(int a, int b, int c)
    {
        return a+b;
    }
    double add(double a, double b)
    {
        return a+b;
    }
}
```

Overloading Rules:

- Method name must be same
- Parameter list must differ (number, type or order)
- Return type alone cannot differ
- Can overload constructors
- Can overload static methods

Method Overriding

Definition: Method overriding occurs when a subclass provides its own implementation of a parent class method.

```
class Parent
{
    void show()
    {
        System.out.println("Parent");
    }
}
class Child extends Parent
{
    @Override
    void show()
    {
        System.out.println("Child");
    }
}
```

```
Parent p = new Child();
p.show() // Child
```



Overriding Rules:

- Method signature must be same
- Access level cannot be reduced
- Return type must be same or covariant
- final methods cannot be overridden
- static methods are hidden, not overridden

Overloading vs Overriding

Feature	Overloading	Overriding
Occurs in	Same Class	Parent-child classes
Parameters	Must differ	Must be same
Return Type	Any (with diff param)	Same/covariant
Binding	Compile-time	Runtime
Inheritance	Not required	Required

Static Method Case

```
class A
{
    static void show() { }
}
class B extends A
{
    static void show() { } //method hiding
}
```

✓ Compiles

✗ Not overriding

Common Pitfalls

- Confusing overloading with overriding
- Expecting runtime polymorphism from overloading
- Trying to override static or final methods

Best Practices

- Use overloading for same behavior, different inputs
- Use overriding for changing behavior in subclass
- Always use @Override annotation

Access Modifiers in Java

Access modifiers control the visibility and access level of classes, variables, methods, and constructors.

Why Access Modifiers Matter?

- Enforce encapsulation
- Protect internal implementation
- Control API exposure
- Improve maintainability

Important Rules

- You cannot reduce visibility with overriding methods
- private methods are not overridden
- Access modifiers apply at compile time

Types of Access Modifiers

Java provides four access levels:

Modifier	Keyword
Public	public
Protected	protected
Default (package-private)	(no keyword)
Private	private

Access Level Hierarchy: public > protected > default > private

Access Scope Table:

Modifier	Same Class	Same Package	Subclass diff pkg	Everywhere
public	✓	✓	✓	✓
protected	✓	✓	✓	✗
default	✓	✓	✗	✗
private	✓	✗	✗	✗

1. Public

- Accessible from anywhere
- Used for APIs and entry points

```
public class Test  
{  
    public int x;  
}
```

2. Protected

- Accessible within same package
- Accessible in subclass (even in different packages)

```
protected void display() {}
```

3. Default (Package-Private)

- No keyword used
- Accessible only within same package

```
void show() {}
```

4. Private

- Accessible only within the same class
- Core of encapsulation

```
private int age;
```

Access Modifiers on Classes

Modifier	Allowed?
public	✓
default	✓
protected	✗
private	✗

Top-level classes can be public or default

Access Modifiers on Members

Member	Allowed Modifiers
Variables	All
Methods	All
Constructors	All
Classes (inner)	All

Best Practices

- Make variables private
- Expose behavior using public methods
- Use protected carefully
- Minimize public API surface

final, static, this, super

These keywords control behavior, scope, and inheritance in Java.

final Keyword

final is used to restrict modification.

1. final Variable

- Value cannot be changed once assigned

`final int MAX = 100;` It must be initialized

2. final Method

- Cannot be overridden

```
class A
{
    final void show() {}
}
```

3. final Class

- Cannot be inherited

`final class Utility {}`

Note: String class is final

Rules for final

- final variable → immutable reference
- final methods → no overriding
- final classes → no inheritance

static Keyword

static belongs to the class, not objects.

1. static Variable

- Shared among all objects

```
class Test
```

```
{
```

```
    static int count = 0;
```

```
}
```

2. static Method

- Can be called without object
- Cannot access instance members directly

```
static void display() {}
```

Minimize static usage

3. static Block

- Executes once when class loads

```
static
```

```
{
```

```
    System.out.println("Static block");
```

```
}
```

Static Memory

- Stored in Method Area
- Single copy per class

this Keyword

this refers to the current object.

Uses of this:

- Differentiate instance variables
- Call current class method
- Call current class constructor

```
this.x = x;  
this();
```

super Keyword

super refers to the parent class object.

Uses of super:

- Access parent class variables
- Call parent class methods
- Call parent class constructor

```
super.x;  
super.show();  
super();
```

this vs super

Feature	this	super
Refers to	Current object	Parent Object
Used in	Same class	Subclass
Calls constructor	✓	✓
Calls methods	✓	✓

Important Rules

- this() and super() must be first statement
- Cannot use both together
- Static methods cannot use this or super

Common Pitfalls

- Assuming final means object immutability
- Overriding static methods (method hiding)
- Using this in static context

Inner Classes in Java

An inner class is a class defined inside another class.

Why Inner Classes Are Used

- Better logical grouping
- Improved encapsulation
- Access to outer class members
- Cleaner, more readable code

Inner Classes

- Member Inner Class
- Static Nested Class
- Local Inner Class
- Anonymous Inner Class

Member Inner Class (Non-static inner class)

```
class Outer  
{
```

```
    int x=10;
```

```
    class Inner
```

```
    {
```

```
        void show()
```

```
        {
```

```
            System.out.println(x);
```

```
        }
```

```
    }
```

```
}
```

Declared inside a class but outside methods.

Characteristics:

- Access to all members of outer class
- Cannot have static members
- Requires outer class object

Static Nested Class

```
class Outer
{
    int x=10;
    static class Inner
    {
        void show()
        {
            System.out.println(x);
        }
    }
}
```

Declared using *static*

Characteristics:

- No outer object required
- Can access only static members
- More memory-efficient

Local Inner Class

```
void display()
{
    class Local
    {
        void show()
        {
            System.out.println("Local class");
        }
    }
    new Local().show();
}
```

Defined inside method or block.

Characteristics:

- Scope limited to the block.
- No access modifiers allowed
- Can access final or effectively final variables

Anonymous Inner Class

```
Runnable r = new Runnable() {  
    public void run()  
    {  
        System.out.println("Running");  
    }  
};
```

A class without a name, created for one-time use.

Characteristics:

- No class name
- Reduces boilerplate
- Used with interfaces or abstract classes
- Often replaced by lambdas (Java 8+)

Anonymous Class vs Lambda

- Lambdas are shorter
- Only for functional interfaces

```
Runnable r = () -> System.out.println("Hi");
```

Important Rules

- Inner classes increase coupling
- Prefer static nested classes when possible
- Avoid deep nesting
- Prefer lambdas for simple behavior